Свёрточные нейронные сети

Softmax и постановка задачи классификации

В прошлой лекции мы обсуждали регуляризацию и нормализацию в нейронных сетях, а ещё раньше — базовую архитектуру MLP и функции активации. Для задач бинарной классификации нам было достаточно сигмоиды, однако для работы с большим количеством классов нужна другая функция, которая превращает произвольный вектор чисел в распределение вероятностей.

Определение 1 (Softmax) Softmax — это функция, которая каждому логиту z_i ставит в соответствие значение

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{1}$$

и тем самым переводит выходы последнего слоя в числа из отрезка $[0, 1]$, сумма которых равна 1.

Именно поэтому softmax почти всегда используется в конце сети для задачи многоклассовой классификации: после него можно интерпретировать выход модели как вероятности принадлежности объекта к различным классам.

> **Замечание**
>
> На практике softmax почти всегда сочетают с кросс-энтропией. Эта связка стала стандартом для задач классификации изображений.

Однако даже если мы научились решать многоклассовую классификацию, остаётся другой вопрос: какую архитектуру использовать, когда на вход подаётся изображение? Оказывается, обычный многослойный перцептрон для этого подходит очень плохо.

© Пицик Харитон, Карасев Вадим, 2026

Обработка изображений. Почему MLP плохо подходит?

Представьте, что мы хотим подать на вход нейронной сети цветную фотографию размера 1024×1024 пикселей. У цветного изображения три канала, поэтому число входных признаков равно

$$D = 1024 \times 1024 \times 3 \approx 3.15 \times 10^6 \tag{2}$$

Если теперь взять скрытый слой размера 4096, то только матрица весов между входом и этим слоем будет иметь размер

$$|W| = 4096 \times 3.15 \times 10^6 \approx 1.29 \times 10^{10}, \tag{3}$$

то есть речь идёт уже о десятках миллиардов параметров только на первом слое. А ведь в реальной сети таких слоёв обычно несколько.

Это приводит сразу к нескольким проблемам.

1. **Слишком много параметров.** Такую модель трудно хранить, трудно обучать и почти невозможно обеспечить достаточным количеством данных.
2. **Высокий риск переобучения.** Если параметров слишком много по сравнению с объёмом данных, сеть быстро запоминает обучающую выборку.
3. **Игнорирование пространственной структуры.** Полносвязный слой рассматривает изображение как длинный вектор и не учитывает, что соседние пиксели тесно связаны друг с другом.

Последний пункт особенно важен. Если на двух изображениях один и тот же объект просто немного сдвинут вправо или влево, с точки зрения содержимого они почти одинаковы. Однако для MLP это уже совершенно другие входные векторы: значения пикселей оказались в новых позициях, и сеть не понимает, что это тот же самый паттерн.

Следовательно, нам нужна архитектура, которая одновременно:

- умеет работать с **локальными** фрагментами изображения;
- переиспользует одни и те же детекторы признаков в разных частях картинки;
- содержит существенно меньше параметров, чем полносвязная сеть.

Именно так и устроены свёрточные нейронные сети.

Свёрточные нейронные сети (CNN)

Идея CNN состоит в том, чтобы обрабатывать изображение **локально**: вместо связи каждого пикселя с каждым нейроном мы рассматриваем небольшие окрестности пикселей, которые часто называют патчами. Это сразу решает две базовые проблемы MLP:

- соседние пиксели начинают анализироваться совместно;
- один и тот же детектор признаков можно применять по всему изображению.

Свёртка

Ключевая операция CNN — это **свёртка** (convolution). Мы берём фильтр размера $k \times k$ и последовательно перемещаем его по изображению. В каждой позиции считается скалярное произведение ядра свёртки и соответствующего патча изображения:

$$(I * K)(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I_{i+m, j+n} \cdot K(m, n) \tag{4}$$

где I — входное изображение, а K — ядро свёртки.

Если вход имеет размер $H \times W$, размер ядра равен $k \times k$, а шаг свёртки равен s, то пространственный размер выхода вычисляется так:

$$H_{\text{out}} = \left\lfloor \frac{H - k}{s} \right\rfloor + 1, \tag{5}$$

$$W_{\text{out}} = \left\lfloor \frac{W - k}{s} \right\rfloor + 1 \tag{6}$$

На практике это означает, что свёртка обычно уменьшает изображение по высоте и ширине.

Padding

Чтобы не терять размер слишком быстро, по краям изображения часто добавляют нули. Это называется **padding**. Если величина дополнения равна p, то формула принимает вид

$$H_{\text{out}} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1 \tag{7}$$

и аналогично для ширины. Например, при $k = 3$, $s = 1$ и $p = 1$ пространственный размер сохраняется.

> **Замечание**
>
> Padding полезен не только для сохранения размерности. Без него граничные пиксели начинают слишком быстро терять влияние на дальнейшие слои.

Разделяемые веса

Главное отличие свёрточного слоя от полносвязного — **shared weights**, или разделяемые веса. Один и тот же фильтр используется для всех позиций изображения. Это означает, что число параметров больше не зависит напрямую от высоты и ширины входа.

Если у входа C_{in} каналов, то один фильтр размера $k \times k$ содержит

$$|K| = k \times k \times C_{\text{in}} \tag{8}$$

параметров. Если же таких фильтров C_{out}, то число весов слоя равно

$$|W_{\text{conv}}| = k \times k \times C_{\text{in}} \times C_{\text{out}} \tag{9}$$

плюс смещения для выходных каналов.

Например, для цветного изображения и ядра 3×3 один фильтр имеет всего $3 \times 3 \times 3 = 27$ параметров — это несопоставимо меньше, чем в полносвязных слоях для изображений высокого разрешения.

Пулинг

После свёртки часто применяют ещё одну операцию — **pooling**. Она уменьшает пространственный размер карты признаков и делает представление более устойчивым к небольшим сдвигам объекта на изображении.

Наиболее популярный вариант — **Max Pooling**. Для каждого окна $p \times p$ берётся максимальное значение:

$$y(i,j) = \max_{0 \le m,n < p} x(i \cdot s + m, j \cdot s + n) \tag{10}$$

У пулинга есть два ключевых эффекта:

- **сжатие** представления;
- **небольшая инвариантность к сдвигу**.

Типичная архитектура CNN

Классическая свёрточная сеть устроена примерно так:

$$\underbrace{[\mathrm{Conv} \to \mathrm{ReLU}] \times N}_{\text{извлечение признаков}} \to \mathrm{Flatten} \to \underbrace{[\mathrm{Linear} \to \mathrm{ReLU}] \times M}_{\text{классификация}} \tag{11}$$

Первые слои выделяют простые локальные структуры, далее признаки становятся всё более абстрактными, а в конце на основе этих признаков работает обычный классификатор.

Если сравнить по количеству параметров первый свёрточный слой с 64 фильтрами 3×3 и первый полносвязный слой для изображения 1024×1024, то различие окажется колоссальным. Свёрточный слой требует порядка тысяч параметров, а MLP — миллиарды.

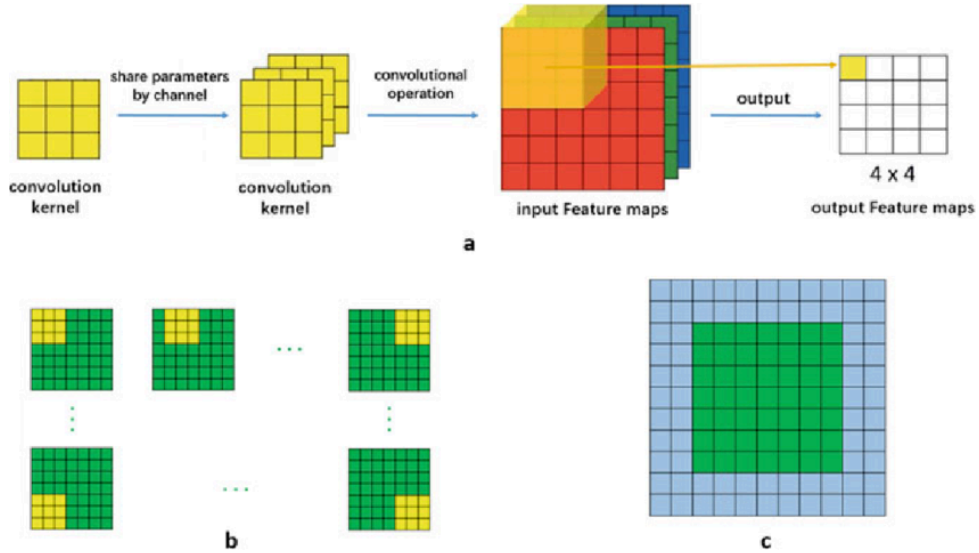

Рис. 1. Многоканальная свёртка как работа с тензорами входа и фильтра

Дополнения к CNN

Многоканальная свёртка

Реальные изображения обычно имеют несколько каналов. Для RGB это три канала, поэтому вход свёрточного слоя имеет размер $H \times W \times C_{\text{in}}$, где чаще всего $C_{\text{in}} = 3$.

Фильтр также становится трёхмерным: его размер равен $k \times k \times C_{\text{in}}$. Один такой фильтр смотрит сразу на все каналы в своей локальной области и в каждой позиции выдаёт одно число. Если мы используем C_{out} фильтров, то на выходе получаем тензор размера

$$H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}} \tag{12}$$

с количеством параметров

$$|W| = k \times k \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}. \tag{13}$$

Интуитивно разные фильтры учатся искать разные признаки. На ранних слоях это обычно края, цветовые переходы и простые текстуры, на средних — части объектов, а на глубоких — уже семантически более сложные конструкции, например глаза, колёса или контуры лиц.

Другие виды пулинга

Хотя Max Pooling исторически наиболее популярен, существует и **Average Pooling**, где по окну берётся не максимум, а среднее:

$$y(i,j) = \frac{1}{p^2} \sum_{0 \le m,n < p} x(i \cdot s + m, j \cdot s + n) \tag{14}$$

Такой вариант слабее выделяет ярко выраженные активации, но иногда даёт более сглаженное представление.

Особый интерес представляет **Global Average Pooling** (GAP). В этом случае усреднение берётся сразу по всей карте признаков каждого канала:

$$\text{GAP}(X)_c = \frac{1}{H \cdot W} \sum_{i,j} X_{i,j,c} \tag{15}$$

Тогда тензор $H \times W \times C$ превращается в вектор длины C. Это позволяет отказаться от огромных полносвязных слоёв в конце сети и сильно сократить количество параметров.

Полный пайплайн обучения CNN

Схематично обучение свёрточной сети выглядит так:

- прямой проход: $X \to [\text{Conv} \to \text{ReLU}] \to \text{Pooling} \to \ldots \to \text{Flatten} \to \text{Linear} \to \text{Softmax} \to \hat{p}$;
- вычисление функции потерь, обычно кросс-энтропии;
- обратное распространение ошибки по всем параметрам;
- обновление весов градиентным спуском или его модификациями.

Недостатки CNN

Несмотря на огромный успех свёрточных сетей, у них есть и ограничения:

1. При сильном пулинге и последовательных свёртках часть пространственной информации неизбежно теряется.
2. Очень глубокие CNN исторически было трудно обучать из-за затухающего градиента.
3. Для хорошего качества обычно требуется много данных и вычислительных ресурсов.

Именно попытки решить эти проблемы и привели к появлению известных архитектур, о которых пойдёт речь в следующей лекции.